REACT INTERVIEW CHEAT SHEET

Quick Guide for Learning + Interview Prep

1. What is React?

React is a JavaScript library for building user interfaces using reusable components.

```
function App() {
  return <h1>Hello React</h1>;
}
```

2. What is a Component?

A component is a reusable piece of UI.

```
function User() {
  return <h2>Vishal</h2>;
}
```

3. What is JSX?

JSX lets us write HTML-like syntax inside JavaScript.

```
const element = <h1>Hello</h1>;
```

Transpiles to:

```
React.createElement("h1", null, "Hello");
```

4. Props vs State

Props

- Passed from parent
- Read-only
- Used to pass data

```
<User name="Vishal" />
```

State

- Managed inside component
- Can change
- Used to manage data

```
const [count, setCount] = useState(0);
```

Props come from parent. State belongs to the component.

5. useState

useState is a Hook that lets us add state.

```
const [count, setCount] = useState(0);
```

- count → current value
- setCount → function to update value
- 0 → initial value

Changing state re-renders the component.

Count: 0 [Increase]

6. useEffect

useEffect is used for side effects.

```
useEffect(() => {
  // side effect logic
}, []);
```

Dependency Array

- No array → runs after every render
- [] → runs once after mount
- [x] → runs when x changes

7. useEffect Cleanup

Used to clean up things like timers, event listeners, subscriptions.

```
useEffect(() => {
  const timer = setInterval(() => {}, 1000);

  return () => {
    clearInterval(timer);
  };
}, []);
```

8. Lists & Keys

Keys help React identify which items changed, added or removed.

```
users.map(user => (
  <User key={user.id} user={user} />
))
```

✔ Use stable IDs as key

✘ Avoid using index as key if list can change.

9. Conditional Rendering

Show different UI based on condition.

```
{isLoggedIn ? <Dashboard /> : <Login />}

{isLoggedIn && <Dashboard />}
```

10. Event Handling

React events are written in camelCase.

```
<button onClick={handleClick}>
  Click
</button>
```

Don't do: onClick={handleClick()}

11. Controlled Component

Input value controlled by React state.

```
const [name, setName] = useState("");

<input
  value={name}
  onChange={e => setName(e.target.value)}
/>
```

12. Component Communication

Parent → Child (Props)

```
<Child name="Vishal" />
```

Child → Parent (Callback)

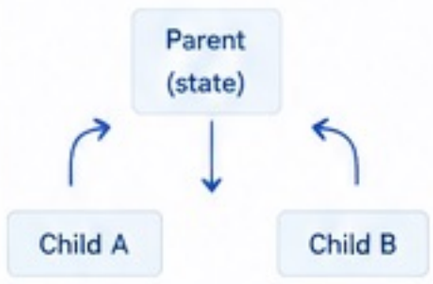
```
<Child onSave={
  handleSave
} />
```

Sibling → Sibling (Lift state up)

Lift state to common parent.

13. Lifting State Up

Move state to the closest common parent.



14. Immutability

Don't modify state directly.

Bad:

```
user.name = "Rahul";
```

✘

Good:

```
setUser({
  ...user,
  name: "Rahul"
});
```

✔

15. Virtual DOM

React keeps a lightweight copy of the UI in memory and updates only what changed.

State Changes → New Virtual DOM → Compare with old → Update only changed parts

16. Re-render

When state or relevant props change, component re-renders.

setState() → Component re-renders → React compares → DOM updates

17. useRef

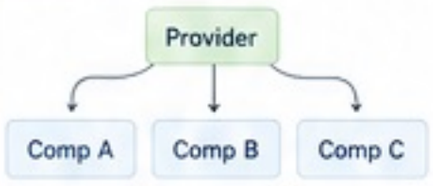
1. Access DOM element
2. Store value without causing re-render

```
const inputRef = useRef();
<input ref={inputRef} />
inputRef.current.focus();
```

Changing ref.current does NOT re-render.

18. Context API

Share data without passing props through every level.



Use for: theme, user, language, settings, etc.

19. useMemo vs useCallback

useMemo — Memoize a value

```
const result =
  useMemo(() =>
  expensiveCalc(data),
  [data]
);
```

useCallback — Memoize a function

```
const handleClick =
  useCallback(() => {
  console.log("Hi");
}, []);
```

useMemo → value | useCallback → function

20. React.memo

Prevents unnecessary re-renders of a component if props didn't change.

```
const User = React.memo(
  function User({ name }) {
    return <h2>{name}</h2>;
  }
);
```

Use for performance optimization.

21. API Calls

Usually done in useEffect.

```
const [users, setUsers] = useState([]);

useEffect(() => {
  fetch('/api/users')
    .then(res => res.json())
    .then(data => setUsers(data));
}, []);
```

Flow: Mount → useEffect → API → setState → Re-render

22. React Router

Used for navigation.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/users" element={<Users />} />
</Routes>
```

Different URL → Different Component

React vs Angular (High Level)

	React	Angular
Type	Library	Framework
Template	JSX	HTML Templates
Flexibility	More Flexible	More Opinionated
Routing	React Router	Built-in Router
State Mgmt	Flexible	Services / RxJS
Learning Curve	Easier to start	Steeper

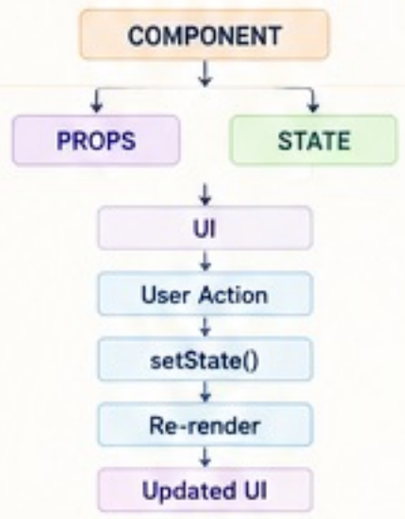
Most Important Hooks (Quick View)

useState	Manage state
useEffect	Side effects
useRef	Reference / value without re-render
useContext	Share data
useMemo	Cached value
useCallback	Cached function

Top 15 Interview Questions (Practice!)

1. What is React?
2. What is JSX?
3. What is a component?
4. Props vs State?
5. What is useState?
6. What happens when state changes?
7. What is useEffect?
8. What does the dependency array in useEffect do?
9. What is a controlled component?
10. Why do we need keys when rendering lists?
11. How does a child communicate with its parent?
12. What is lifting state up?
13. Why shouldn't we directly modify state?
14. useMemo vs useCallback?
15. React vs Angular?

Remember This Mental Model



Golden Rules

- ✓ Don't modify state directly.
- ✓ Always use the setter.
- ✓ Keep components small and reusable.
- ✓ Understand the data flow.

Bonus Tips

- ★ Practice by building small projects.
- ★ Read code, understand flow.
- ★ Write code from memory.
- ★ Explain concepts out loud.

Focus Order (Use your 1 hour wisely)

Must Know (60%) → Basics, State, Props, useEffect, Lists, Forms, Communication, API

Good to Know (30%) → useRef, Context, Lifting State, Router, Conditional Rendering

Nice to Know (10%) → useMemo, useCallback, React.memo (concept level)

HAPPY CODING! 🚀

ANGULAR INTERVIEW CHEAT SHEET

Quick Guide for Learning + Interview Prep

1. What is Angular?

Angular is a TypeScript-based open-source front-end framework for building dynamic, single-page applications (SPAs) with rich UI. It supports two-way data binding, dependency injection, modular architecture, routing, and testing.

TypeScript • Components
Data Binding • DI • Modules

2. What are Directives?

Directives are special markers in the DOM that allow Angular to manipulate elements, attributes, and components.
Angular identifies them by the prefix ng-.

Types of Directives

1. **Component Directives**
   (Has view + logic)
   `@Component()`
2. **Structural Directives**
   (Alter DOM structure, prefixed *)
   `*ngIf, *ngFor`
3. **Attribute Directives**
   (Change appearance/behavior)
   `ngClass, ngStyle`

3. What is a Component?

Components are the building blocks of an Angular application. Each component contains HTML (template), CSS (styles) and TypeScript (logic).

```
@Component({
  selector: 'app-user',
  template: `
    <h3>{{ title }}</h3>`
  ,
  styleUrls: ['./user.component.css']
})
export class UserComponent {
  title = 'Hello Angular!';
}
```

4. What is a Module?

Modules (@NgModule) group related components, directives, pipes and services together. AppModule is the root module that bootstraps the application.

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

5. What is TypeScript & Why do we need it?

TypeScript is a superset of JavaScript that adds static typing, classes, interfaces and more.

Angular uses it because it:

- Catches errors early (type safety)
- Supports OOP (classes, interfaces)
- Better tooling (autocompletion, refactoring, navigation)

6. Importance of Angular CLI

- ✓ Scaffolds projects quickly
- ✓ Generates components, services, modules, etc.
- ✓ Builds, serves and deploys apps
- ✓ Runs unit & E2E tests easily
- ✓ Optimized production builds (AOT, minification, tree shaking)

Common Commands

`ng new my-app`	Create project
`ng generate component cmp`	Generate component
`ng serve -o`	Run app
`ng build --prod`	Production build
`ng test`	Run unit tests

7. package.json

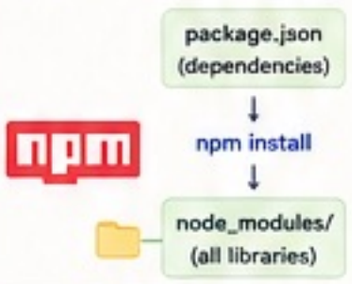
It is essential as it:

- ✓ Lists all dependencies
- ✓ Contains project metadata (name, version, description)
- ✓ Defines scripts (build, test, serve)
- ✓ Maintains versioning of packages

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "ng": "ng",
    "start": "ng serve"
  },
  "dependencies": {
    "@angular/core": "^17.0.0" }
}
```

8. NPM & node_modules

- NPM (Node Package Manager): Default package manager for Node.js used to install and manage libraries and packages.
- node_modules folder: Contains all dependencies listed in package.json required to run, build and maintain the project.



9. Data Binding in Angular

Angular supports different types of data binding.

Type	Syntax
Interpolation	{{ data }}
Property Binding	[property]="data"
Event Binding	(event)="onEvent()"
Two-way Binding	[(ngModel)]="data"

Example:

```
<input [(ngModel)]="name" />
<p>{{ name }}</p>
```

10. Dependency Injection (DI)

A design pattern where Angular provides dependencies (services) to components or other services.

```
@Injectable({
  providedIn: 'root'
})
export class UserService {
  getUsers() { /* ... */ }
}
```

- Reduces tight coupling
- Easy testing (mock services)
- Greater reusability

11. Services in Angular

Services are used to share data or logic across components.

```
@Injectable({ providedIn: 'root' })
export class DataService {
  data = 'Hello from Service!';
}
```

Usage:

```
constructor(private dataService: DataService) {}
ngOnInit() {
  console.log(this.dataService.data);
}
```

12. Angular Lifecycle Hooks

Lifecycle hooks are methods that Angular calls at different stages.

Hook	Called When
ngOnChanges	When @Input changes
ngOnInit	After component init
ngDoCheck	On every change detection run
ngAfterContentInit	After content projected
ngAfterViewInit	After view initialized
ngOnDestroy	Before component destroyed

Helps in writing efficient and controlled logic.

13. Routing in Angular

Angular Router enables navigation between different views.

```
const routes: Routes = [
 { path: 'home', component: HomeComponent },
 { path: 'about', component: AboutComponent }
];
```

```
<router-outlet></router-outlet>
```

- RouterModule.forRoot(routes)
- <a routerLink="/home">Home
- Lazy loading improves performance

14. Pipes in Angular

Pipes transform data in templates.

```
{{ today | date: 'dd/MM/yyyy' }}
{{ price | currency:'INR' }}
{{ text | uppercase }}
```

Types of Pipes

- Built-in Pipes: date, currency, uppercase, lowercase, json, async
- Custom Pipes: Create your own using @Pipe()

15. Forms in Angular

Angular supports two types of forms.

1. Template-driven Forms (Simple forms)

```
<form #f="ngForm">
 <input name="name" ngModel />
</form>
```

2. Reactive Forms (Complex forms)

```
this.form = this.fb.group({
  name: ['']
});
```

Reactive forms are more scalable and testable.

16. Observables & RxJS

Angular uses RxJS for handling asynchronous data streams.

```
this.http.get('api/data').subscribe({
  next: (res) => console.log(res),
  error: (err) => console.error(err)
});
```

Common Operators:

map | filter | switchMap | mergeMap | catchError | tap

17. HTTP Client

Used to make HTTP requests.

```
constructor(private http: HttpClient) {}

this.http.get('api/users')
  .subscribe(data => this.users = data);
```

Methods:

GET | POST | PUT | DELETE

18. Angular Decorators

Decorators provide metadata.

@Component	Defines a component
@NgModule	Defines a module
@Injectable	Defines a service
@Input()	Pass data to child
@Output()	Send data to parent
@HostListener()	Listen to host events

19. Change Detection

Angular checks for changes in data and updates the view.

- Default Strategy: Checks entire component tree.
- OnPush Strategy: Checks only when @Input changes or an event occurs.

Use OnPush for better performance in large applications.

20. Testing in Angular

Angular has great built-in testing support.

- Unit Testing: Jasmine + Karma
- E2E Testing: Protractor / Cypress

Run Tests

```
ng test
ng e2e
```

Angular vs React (High Level)

Feature	Angular	React
Type	Framework	Library
Language	TypeScript	JavaScript (JSX)
Architecture	Opinionated	Flexible
Data Binding	Two-way	One-way
Routing	Built-in Router	React Router (external)
State Mgmt	RxJS + Services	Various (Context, Redux)
Learning Curve	Steeper	Easier to start

Most Important Angular Concepts (Quick View)

Components	Building blocks of UI
Modules	Organize code
Services	Share logic & data
Dependency Injection	Provide dependencies
Data Binding	Connect data with UI
Routing	Navigate between views
Pipes	Transform data
Forms	Handle user input
Observables	Handle async data

Top 15 Angular Interview Questions (Practice!)

1. What is Angular?
2. What are directives and its types?
3. What is a component?
4. What is a module?
5. What is the package.json file?
6. What is Angular CLI and its importance?
7. What is data binding in Angular?
8. What is dependency injection?
9. What are services in Angular?
10. What are lifecycle hooks?
11. What is routing and how does it work?
12. Template-driven vs Reactive forms?
13. What are pipes and their types?
14. What is Observable in Angular?
15. What is change detection?

Angular Interview Flow (Mental Model)

User Action
↓
Event Binding
↓
Component Method
↓
Update Data / Service Call
↓
Change Detection
↓
Update View (UI)

Golden Rules

- ✓ Keep components small & reusable
- ✓ Use services for shared logic
- ✓ Prefer reactive forms for complex scenarios
- ✓ Unsubscribe from observables
- ✓ Lazy load modules for performance

Bonus Tips

- ★ Use Angular CLI commands
- ★ Follow feature module structure
- ★ Use OnPush change detection
- ★ Write unit tests for critical logic
- ★ Keep code clean & modular

Focus Order (1 Hour Plan)

- (25%) Basics: Components, Module, CLI, Directives
- (20%) Data Binding, DI, Services
- (20%) Routing, Forms, Pipes
- (20%) Observables, HTTP, Lifecycle Hooks
- (15%) Testing, Change Detection, Best Practices

Remember

"Understand the concepts, write clean code, and practice building small Angular apps."

Apache Kafka

A distributed event streaming platform

What is Kafka?

Apache Kafka is a distributed, high-throughput, fault-tolerant, real-time data streaming platform. It allows applications to publish (produce) and subscribe (consume) streams of records (events) in a scalable and reliable way.

High Throughput | Fault Tolerant | Horizontally Scalable

Key Concepts

Topic	A category or feed name to which records are published.
Partition	A topic is split into one or more partitions for parallelism and scalability.
Offset	A unique ID for each record within a partition.
Producer	Application that publishes records to a topic.
Consumer	Application that subscribes to a topic and processes records.
Broker	Kafka server that stores data and serves clients.
Consumer Group	A group of consumers that work together to consume a topic.

How Kafka Works (High Level)



- Records are appended to partitions in order.
- Partitions are replicated across brokers for fault tolerance.
- Consumers read records at their own pace using offsets.

Why Use Kafka? (Benefits)

- Handles large volume of real-time data
- Decouples systems and microservices
- Durable & reliable (data is persisted)
- Scalable and elastic
- High availability with replication
- Supports stream & batch processing
- Supports multiple consumers (fan-out)

Common Use Cases

Real-time Data Pipelines
Ingest data from multiple sources, stream it, and load into databases/data lakes.

Event-driven Microservices
Services publish events to Kafka and other services consume them asynchronously.

Log & Metrics Aggregation
Collect logs, metrics, and monitor data from many servers in real time.

Stream Processing
Process events in real time using Kafka Streams, Flink, or Spark.

Change Data Capture (CDC)
Capture DB changes and stream them to other systems in real time.

Notifications & Activity Feeds
Build real-time notifications, feeds, and user activity tracking systems.

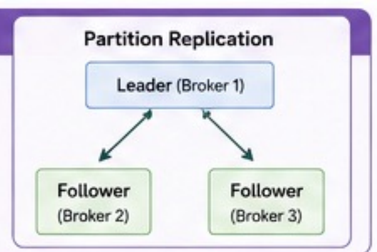
Ecosystem / Integrations

Kafka Connect Data integration	**Kafka Streams** Stream processing	**ksqlDB** Stream SQL engine	**Schema Registry** Schema management
Apache Flink Stream processing	**Apache Spark** Batch & stream	**Prometheus** Monitoring	**Grafana** Visualization

Kafka can integrate with databases, data lakes, BI tools, monitoring tools, and many other systems.

Architecture & Durability

- **Distributed:** Data is distributed across multiple brokers.
- **Replication:** Each partition is replicated across brokers.
- **Durable:** Data is persisted to disk.
- **Retention:** Data is retained for configurable time or size.



Summary

Kafka is the backbone of modern real-time data systems. It enables decoupled, scalable, and reliable event streaming for a wide range of applications across industries.

In Short

Producers generate events → Kafka stores events in topics → Consumers process events → Systems stay in sync in real time

Popular Industries Using Kafka

E-commerce | Banking & Finance | Healthcare | Telecom | Media & Entertainment | IoT & Manufacturing

GraphQL

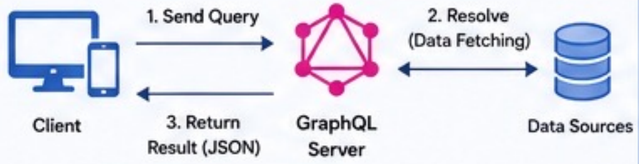
A query language for your API

What is GraphQL?

GraphQL is a query language for APIs and a runtime for executing those queries with your existing data. It provides a complete and understandable description of the data in your API, gives clients the power to ask for exactly what they need and nothing more.

- **Declarative** (Describe what you need)
- **Precise** (Get exactly what you ask)
- **Powerful** (Nested queries, relationships)
- **Efficient** (One request, less over/under fetching)

How GraphQL Works



1. Client sends a GraphQL query (specifying the data it needs).
2. GraphQL server validates the query against the schema.
3. Resolvers fetch the data from databases, services, or APIs.
4. Server returns exactly the requested data in JSON.

Ecosystem / Tools

Apollo Server & Client	GraphQL Java	graphQL-js (JavaScript)	Relay (Facebook)	Hasura (GraphQL Engine)
Prisma	GraphiQL (IDE)	Altair (IDE)	Postman (Testing)	Apollo Studio (Explorer)

Many client libraries available for JavaScript, Swift, Kotlin, Dart, Python, Go and more.

Key Concepts

Schema	The contract of your API. Defines the types, queries, mutations and relationships.
Type	Represents an object with fields. (e.g., User, Post, Comment)
Query	Read-only operation to fetch data.
Mutation	Write operation to create, update or delete data.
Field	A unit of data on a Type. Can be scalar or another Type.
Resolver	Function that resolves a field and returns the data.
Arguments	Inputs you pass to fields to fetch specific data.
Introspection	Built-in capability to query the schema itself.

Example

GraphQL Query

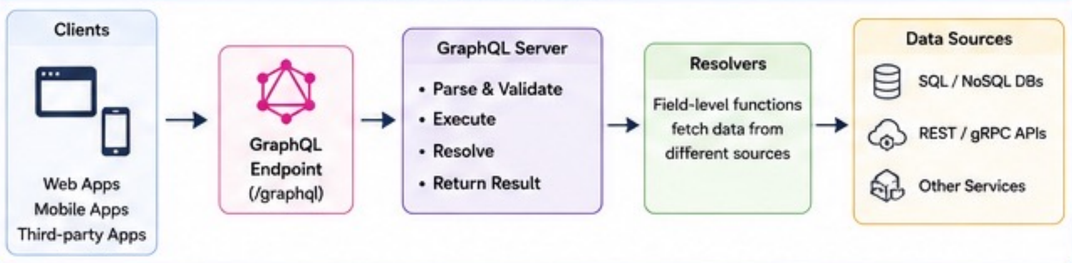
```
query GetUser($id: ID!) {
 user(id: $id) {
   id
   name
   email
   posts {
     id
     title
     createdAt
   }
 }
}
```

JSON Response

```
{
 "data": {
  "user": {
   "id": "1",
   "name": "Alice",
   "email": "alice@example.com",
   "posts": [
    { "id": "101", "title": "Hello World", "createdAt": "2024-05-20" },
    { "id": "102", "title": "GraphQL Rocks", "createdAt": "2024-05-21" }
   ]
  }
 }
}
```

Client asked for specific fields → Server returned only those fields.

Architecture Overview



Benefits of GraphQL

- Fetch only the data you need (no over-fetching)
- Reduce the number of network requests
- Strongly typed schema & self-documenting
- Easier API evolution & backward compatibility
- Real-time capabilities with subscriptions
- Great developer experience & tooling
- One endpoint for multiple clients (web, mobile, etc.)

Common Use Cases

Mobile & Web Apps
Fetch only what's needed for each screen, reduce payload size and improve performance.

Microservices Integration
Aggregate data from multiple services through a single GraphQL API.

Real-time Applications
Use Subscriptions for live updates (chats, notifications, dashboards).

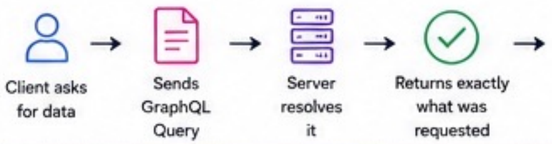
CMS & Content Delivery
Flexible content fetching for blogs, e-commerce, marketing sites.

B2B & Partner APIs
Expose a single, flexible API to multiple internal and external consumers.

Best Practices

- Design a clear and consistent schema.
- Use meaningful names for types and fields.
- Implement pagination for large lists.
- Leverage DataLoader to avoid N+1 queries.
- Secure your schema (depth, complexity, auth).
- Version your API when needed.
- Use caching (persisted queries, CDN).

In Short



Industries Using GraphQL

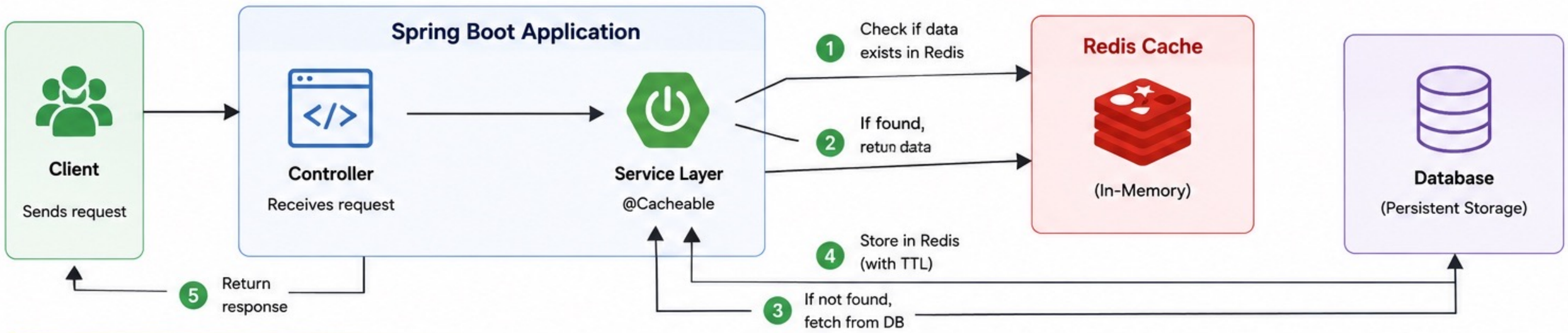
E-commerce · Media & Entertainment · FinTech & Banking · Healthcare · SaaS Platforms · Travel & Hospitality · IoT & Telematics

GraphQL vs REST (At a Glance)

GraphQL	REST
• One endpoint	• Multiple endpoints
• Client decides data shape	• Server decides data shape
• No over/under fetching	• Over/under fetching
• Strongly typed schema	• Schema not enforced

Redis Caching in Spring Boot

Improve performance by storing frequently accessed data in Redis (in-memory cache)



How it works (Cache-Aside Pattern)

1. Application checks Redis for the requested data.
2. If data exists (cache hit) → return it immediately.
3. If data does not exist (cache miss) → fetch from DB.
4. Store the data in Redis with an expiration time (TTL).
5. Return the data to the client.

Benefits

- Reduces database load
- Improves response time
- In-memory speed (super fast)
- Scalable and cost-effective
- Supports TTL for automatic expiration

Example in Spring Boot

```
@Service
public class ProductService {

   @Autowired
   private ProductRepository repositiory;

   @Cacheable(value = "products", key = "#id")
   public Product getProductById(Long id) {
     // This method will be cached in Redis
     return repository.findById(id)
        .orElseThrow(() -> new RuntimeException("Not found"));
   }
}
```

`@Cacheable`: Stores the result in Redis.
`key`: Unique key for the cached data.

Application Configuration

```
spring:
  data:
    redis:
      host: localhost
      port: 6379
      password:
      timeout: 2000ms

  cache:
    type: redis
    redis:
      time-to-live: 600000  # 10 minutes TTL
```

Important Annotations

`@Cacheable`	– Cache the method result.
`@CachePut`	– Update cache without condition.
`@CacheEvict`	– Remove entry from cache.

Use Redis caching for frequently read data that doesn't change often.
It dramatically improves performance and scalability of your Spring Boot application.